

BEIRUT ARAB UNIVERITY

FACULTY OF SCIENCE

DEPARTMENT OF COMPUTER SCIENCE

CMPS347 PROJECT

Supervised by: Dr. Cerine Tafran

And

T.A. Ahmad Ramadan



Ticket Reservation System

Nour Ajami – 202302485

Rayan Ajami – 202302484

Sadeq Sleiman – 202302840

Jad Amer – 202301653

CONTENTS

1. **Abstract**
2. **Introduction**
3. **Technologies Used**
4. **UML Diagram**
5. **Data Structure Concepts Used**
   - 4.1 Linked List
   - 4.2 Node
   - 4.3Priority Queue
   - 4.4 Stack
6. **UML Diagram Explanation**
   - 5.1 Reservation Class
   - 5.2 Flight Class
   - 5.3 Customer Class
   - 5.4 Node Class
   - 5.5 ReservationLinkedList Class
   - 5.6 FlightReservationSystem Class
7. **Conclusion**
8. **Full Code**
9. **Output**

1. Abstract

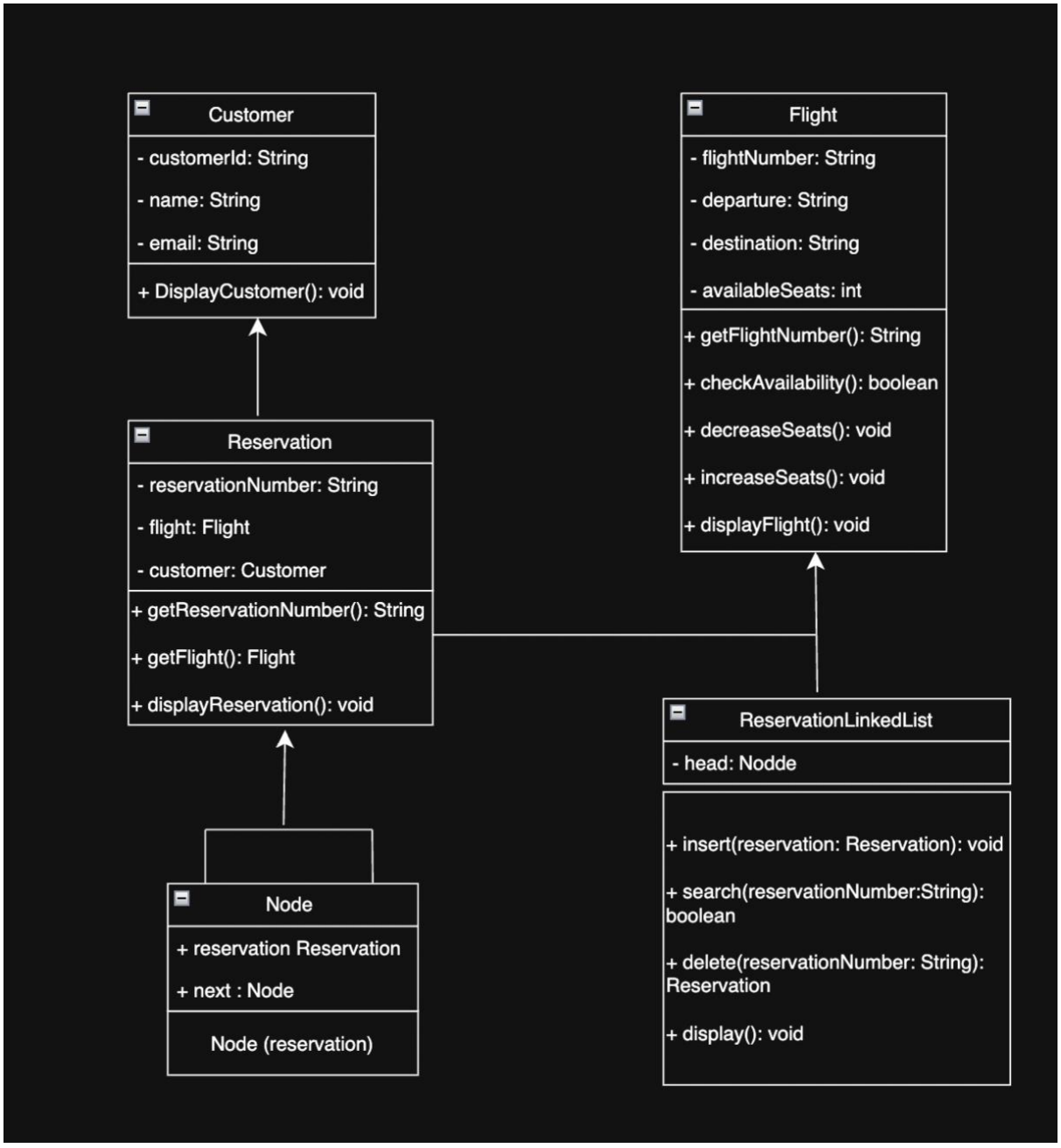
This project is a Java-based Flight Reservation System. It allows users to view available flights, make and cancel reservations, and track cancelled or existing bookings. A custom linked list is used to manage reservations, a priority queue to handle important bookings, and a stack to track canceled reservations.

2. Introduction

The Flight Reservation System is designed to simplify and manage flight bookings. It lets customers do a lot of things, like searching for available flights, booking and cancelling, finding specific reservations, and keeping track of cancelled reservations. To impersonate real-world booking administration, the system makes use of basic programming ideas including OOP and Data Structures. Reservation information is stored and arranged using a custom linked list, and high-priority reservations are handled using a priority queue. To facilitate the retrieval of cancelled reservation, a stack was used. This project replicates the usefulness of a real-world system in a simple manner.

3. UML Diagram

The UML diagram below shows the Flight Reservation System's structure:



4. Concepts Used:

4.1 Linked List

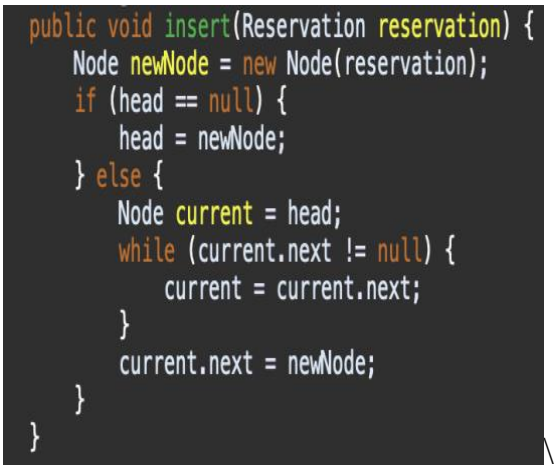
A linked list is a linear data structure where each elements (node) contains a data value and a reference to the next node in the sequence. Unlike arrays, Linked List allow insertion and deletion without shifting elements.

Usage in code:

The ReservationLinkedList class maintains all the reservations. It uses a Node class to link reservations together. This method allows the user to add, delete, or search reservations without the use of arrays.

One Example:

In order to add a new reservation at the end of the list, the insert() method is used by traversing from the head to the last node and linking the new node.



4.2 Node

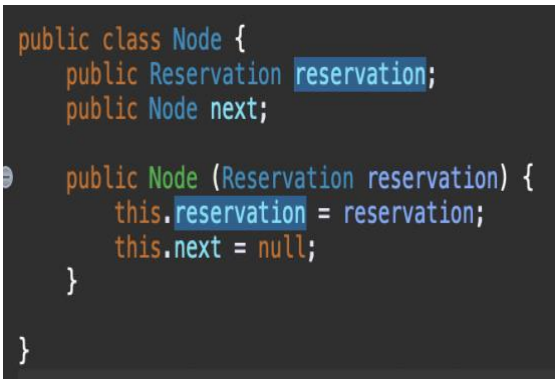
Node is the basic unit of a Linked List. Every node has a reference (pointer) to the following node in the list as well as a reference to the data.

Usage in code:

The Node class is used inside the ReservationLinkedList to store and manage a reservation object and a reference (next) to the next node.

One Example:

When a new reservation is added using the insert() method, a new Node is created, and it holds a Reservation object. This node is then linked to the last node in the list.



```
public class Node {
    public Reservation reservation;
    public Node next;

    public Node (Reservation reservation) {
        this.reservation = reservation;
        this.next = null;
    }

}
```

4.3 Priority Queue

A data structure's priority queue is an extension of a linear queue with the following characteristics: Each component is given a certain priority. This queue's components must all be equivalent. The element with a higher priority will be deleted before the element with a lower priority.

Usage in code:

It is used in FlightReservationSystem for high-priority reservation, where longer reservation numbers have higher priority.

One Example:

```
PriorityQueue<Reservation> priorityQueue = new PriorityQueue<>(
        (r1, r2) -> Integer.compare(r2.getReservationNumber().length(), r1.getReservationNumber().length()));
```

4.4 Stack

A data structure that follows to the Last-In-First-Out (LIFO) concept is called a stack. The first element to be eliminated is the one that was added last.

Usage in code:
Stack is used to store canceled reservation. This makes it easy to view the most recent canceled booking. When a reservation is canceled, it is removed from the linked list and pushed onto the stack.

One Example:

```
System.out.print("Enter Reservation Number to cancel: ");
String cancelResNumber = scanner.nextLine();

Reservation removed = reservationList.delete(cancelResNumber);
if (removed != null) {
    canceledReservations.push(removed);
    removed.getFlight().increaseSeats();
    System.out.println("Reservation cancelled successfully!");
} else {
    System.out.println("Reservation not found.");
}
```

5. UML Diagram Explanation

5.1 Reservation Class

Purpose: The Reservation class is used to manage and store the details of a flight reservation. It ties together a flight and a customer for the reservation process.

Attributes:

- reservationNumber: A unique identifier for each reservation. This could be a string that the system uses to track reservations.
- flight: This is a reference to a Flight object, which represents the specific flight being reserved.
- customer: This is a reference to a customer object, which represents the customer making the reservation.

Methods:

- getReservationNumber(): This method returns the reservation number of the current reservation.
- displayReservation(): This method prints out all details of the reservation, including the reservation number, flight information, and customer details.

How it works: When a reservation is created, it connects a specific flight to a customer. The class ensures that the Flight object decreases the number of available seats when a reservation is made.

5.2 Flight Class

Purpose: The Flight class stores details about a flight, such as its number, departure and destination locations, and the number of available seats.

Attributes:

- flightNumber: A unique identifier for each flight, typically a string (e.g., “AA101”).
- departure: A string that stores the departure location (e.g., “ISTANBUL”).
- destination: A string that stores the arrival location (e.g., “BANGKOK”).
- availableSeats: An integer that tracks how many seats are available on the flight.

Methods:

- getFlightNumber(): This method returns the flight
- checkAvailability(): This method checks if there are available seats on the flight (returns true if seats are available).
- decreaseSeats(): This method decreases the number of available seats by one every time a reservation is made.
- displayFlight(): This method prints out the details of the flight, including flight number, departure, destination, and available seats.

How it works: When a reservation is made, this class handles checking the availability of seats. It ensures that when a reservation is confirmed, the number of available seats for that flight decreases by one.

5.3 Customer Class

Purpose: The Customer class represents the customer who is making a reservation. It stores the customer's details like their ID, name, and email.

Attributes:

- customerId: A unique identifier for the customer, typically a string.
- name: The name of the customer.
- email: The email address of the customer.

Methods:

- getCustomerId(): This method returns the customer's unique ID.
- displayCustomer(): This method prints out the customer's details, including their ID, name, and email.

How it works: This class ensures that customer details are stored and displayed whenever needed. When making a reservation, this class provides the customer's information, which is then tied to the specific flight reservation.

5.4 Node Class

Purpose: Acts as the building block of the linked list that stores reservations.

Attributes:

- reservation: A Reservation object stored in the node.
- next: References to the next node in the list.

Methods:

- Node (Reservation reservation): Initializes a node with a reservation and sets next to null.

How it works: Used inside ReservationLinkedList to form a chain of reservations.

5.5 ReservationLinkedList Class

Purpose: The ReservationLinkedList class is used to manage the collection of reservations. It acts as a container that stores multiple reservations and supports various operations such as insertion, searching, deletion, and displaying all reservations.

Attributes:

- head: A reference to the first node in the linked list. It acts as the starting point for traversing through all reservations in the list.

Methods:

- insert(Reservation reservation): Adds a new reservation to the linked list. It creates a new node for the reservation and places it at the end of the list.
- search(String reservationNumber): Searches the list for a reservation with the given reservation number. It returns true if the reservation is found, otherwise false.
- delete(String reservationNumber): Deletes a reservation with the specified reservation number from the linked list.
- display(): Iterates through the linked list and displays all reservations in the system.

How it works: The ReservationLinkedList class provides functionality for dynamically adding, removing, and displaying reservations using a linked list structure. Each Node contains a Reservation and points to the next node, allowing easy traversal.

5.6 FlightReservationSystem Class

Purpose: The FlightReservationSystem class is the main driver of the system. It handles the user interface, processes user input, and orchestrates the system's operations, including displaying flights, making reservations, searching, canceling, and displaying all reservations.

Methods:

- main (String[] args): The main method that runs the system. It provides a menu-driven interface, where the user can choose to perform actions, such as viewing flights, making a reservation, or managing existing reservations.

How it works: This class is the entry point of the system. It handles user input through a simple text-based menu and calls the appropriate methods from other classes (such as making a reservation or searching for an existing reservation). It allows users to interact with the flight reservation system, making it user-friendly and functional.

6. Conclusion

In conclusion, the Flight Reservation System project has been a valuable learning experience in both OOP and data structure implementation.
By using a linked list to manage reservations, we were able to understand how dynamic data can be efficiently handled. The project also helped us learn how to organize different parts of the system into separate classes, such as Flight, Customer, and Reservation, each with its own responsibility, which made the system more modular and easier to understand. Through this project, we gained hands-on experience with key programming concepts, and it helped us see how data structures works to create a functional system.
Overall, it was an engaging and practical way to apply what we have learned about programming and problem-solving in CMPS347 Course.

7. Full Code

```
public class Node {
	public Reservation reservation;
	public Node next;

	public Node (Reservation reservation) {
		this.reservation = reservation;
		this.next = null;
	}

}
public class Customer {
	private String customerId;
	private String name;
	private String email;

	public Customer(String customerId, String name, String email) {
		this.customerId = customerId;
		this.name = name;
		this.email = email;
	}

	public void displayCustomer() {
		System.out.println("Customer ID: " + customerId + ", Name: " + name +
", Email: " + email);
	}
```

```
}
public class Flight {
	private String flightNumber;
	private String departure;
	private String destination;
	private int availableSeats;

	public Flight(String flightNumber, String departure, String destination, int
availableSeats) {
		this.flightNumber = flightNumber;
		this.departure = departure;
		this.destination = destination;
		this.availableSeats = availableSeats;
	}

	public String getFlightNumber() {
		return flightNumber;
	}

	public boolean checkAvailability() {
		return availableSeats > 0;
	}

	public void decreaseSeats() {
		if (availableSeats > 0) {
			availableSeats--;
		}
	}

	public void increaseSeats() {
```

```
                availableSeats++;
        }

        public void displayFlight() {
                System.out.println("Flight Number: " + flightNumber + ", Departure: " +
departure + ", Destination: " + destination + ", Available Seats: " + availableSeats);
        }

}
public class Reservation {
        private String reservationNumber;
        private Flight flight;
        private Customer customer;

        public Reservation(String reservationNumber, Flight flight, Customer
customer) {
                this.reservationNumber = reservationNumber;
                this.flight = flight;
                this.customer = customer;
                flight.decreaseSeats();
        }

        public String getReservationNumber() {
                return reservationNumber;
        }

        public Flight getFlight() {
                return flight;
        }
```

```
        public void displayReservation() {
                System.out.println("Reservation Number: " + reservationNumber);
                customer.displayCustomer();
                flight.displayFlight();
        }

}
public class ReservationLinkedList {

        private Node head;

        public ReservationLinkedList() {
                head = null;
        }

        public void insert(Reservation reservation) {
                Node newNode = new Node(reservation);
                if (head == null) {
                        head = newNode;
                } else {
                        Node current = head;
                        while (current.next != null) {
                                current = current.next;
                        }
                        current.next = newNode;
                }
        }

        public boolean search(String reservationNumber) {
                Node current = head;
```

```
                while (current != null) {
                        if
(current.reservation.getReservationNumber().equals(reservationNumber)) {
                                return true;
                        }
                        current = current.next;
                }
                return false;
        }

        public Reservation delete (String reservationNumber) {
                if (head == null) {
                        return null;
                }
                if
(head.reservation.getReservationNumber().equals(reservationNumber)) {
                        Reservation deleted = head.reservation;
                        head = head.next;
                        return deleted;
                }
                Node current = head;
                while (current.next != null) {
                        if
(current.next.reservation.getReservationNumber().equals(reservationNumber)) {
                                Reservation deleted = current.next.reservation;
                                current.next = current.next.next;
                                return deleted;
                        }
                        current = current.next;
                }
```

```
                return null;
        }

        public void display() {
                if (head == null) {
                        System.out.println("No reservations found.");
                        return;
                }
                Node current = head;
                while (current != null) {
                        current.reservation.displayReservation();
                        current = current.next;
                }
        }

}

import java.util.PriorityQueue;
import java.util.Scanner;
import java.util.Stack;

public class FlightReservationSystem {
   public static void main(String[] args) {
            Scanner scan = new Scanner(System.in);
            ReservationLinkedList reservationList = new ReservationLinkedList();

            PriorityQueue<Reservation> priorityQueue = new PriorityQueue<>(
             (r1, r2) -> Integer.compare(r2.getReservationNumber().length(),
r1.getReservationNumber().length())
       );
```

```
Stack<Reservation> canceledReservations = new Stack<>();

Flight flight1 = new Flight ("MEA101", "BEIRUT", "KUWAIT CITY", 5);
Flight flight2 = new Flight ("TK102", "ISTANBUL", "BANGKOK", 3);

while(true) {
    System.out.println("\n=== Flight Reservation System ===");
    System.out.println("1. Display Flights");
    System.out.println("2. Make a Reservation");
    System.out.println("3. Search Reservation");
    System.out.println("4. Cancel Reservation");
    System.out.println("5. Display All Reservations");
    System.out.println("6. Display Canceled Reservations");
    System.out.println("7. Exit");
    System.out.print("Choose an option: ");
    int choice = scan.nextInt();
    scan.nextLine();

    switch (choice) {
    case 1:
        flight1.displayFlight();
        flight2.displayFlight();
        break;

    case 2:
        System.out.println("Enter Reservation Number: ");
        String reservationNumber = scan.nextLine();

        System.out.println("Available Flights:");
```

```
                flight1.displayFlight();
                flight2.displayFlight();

                System.out.println("Choose a flight (1 or 2): ");
                int flightChoice = scan.nextInt();
                scan.nextLine();

                Flight selectedFlight;
                if (flightChoice == 1) {
                        selectedFlight = flight1;
                } else {
                        selectedFlight = flight2;
                }

                if (!selectedFlight.checkAvailability()) {
                        System.out.println("No Seats Available.");
                        break;
                }

                System.out.println("Enter Customer ID: ");
                String customerId = scan.nextLine();
                System.out.println("Enter Customer Name: ");
                String customerName = scan.nextLine();
                System.out.println("Enter Customer Email: ");
                String customerEmail = scan.nextLine();

                Customer customer = new Customer(customerId, customerName,
customerEmail);
                Reservation newReservation = new Reservation(reservationNumber,
selectedFlight, customer);
```

```
            reservationList.insert(newReservation);
            priorityQueue.add(newReservation);

            System.out.println("Reservation made successfully!");
            newReservation.displayReservation();
            break;

        case 3:
            System.out.println("Enter Reservation Number to search: ");
            String searchNumber = scan.nextLine();

            if (reservationList.search(searchNumber)) {
                System.out.println("Reservation found!");
            } else {
                System.out.println("Reservation not found.");
            }
            break;

        case 4:
            System.out.println("Enter Reservation Number to cancel: ");
            String cancelResNumber = scan.nextLine();

            Reservation removed = reservationList.delete(cancelResNumber);
            if (removed != null) {
                canceledReservations.push(removed);
                removed.getFlight().increaseSeats();
                System.out.println("Reservation not found.");
            } else {
                System.out.println("Reservation not found.");
```

```
                }
                break;

            case 5:
                System.out.println("\nAll Reservations: ");
                reservationList.display();
                break;

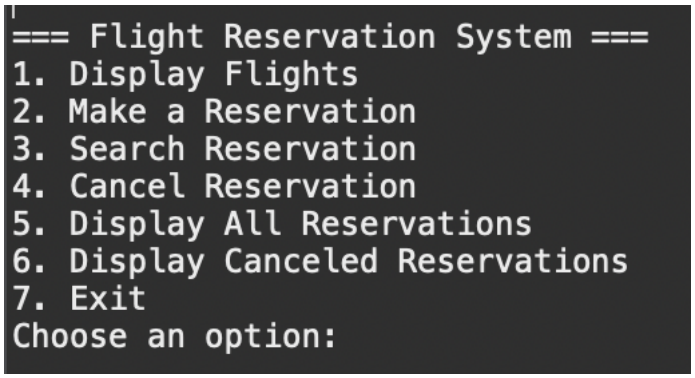
            case 6:
                System.out.println("\nCanceled Reservations:");
                if (canceledReservations.isEmpty()) {
                    System.out.println("No caneled reservations.");
                } else {
                    for (Reservation canceled : canceledReservations) {
                        canceled.displayReservation();
                    }
                }
                break;

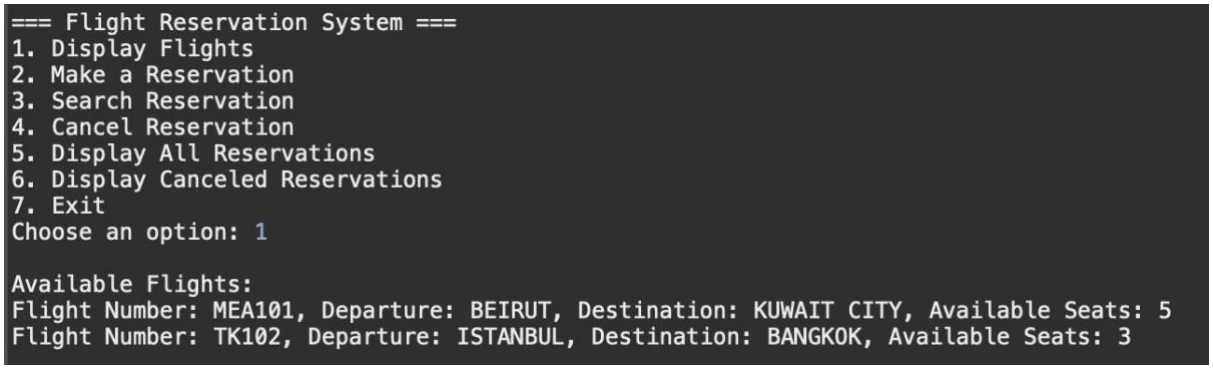
            case 7:
                System.out.println("Exiting the system....");
                scan.close();
                return;

            default:
                    System.out.println("Invalid choice! Please try again.");
             }
        }
    }
}
```

10. Output

Once we run the code this is what is shown. The user can choose the above options and reserve their flight ticket.





Here the user chooses 1 to display the available flights. It shows that there are 2 available flights, their flight number, destination, departure, and available seats.

```
=== Flight Reservation System ===
1. Display Flights
2. Make a Reservation
3. Search Reservation
4. Cancel Reservation
5. Display All Reservations
6. Display Canceled Reservations
7. Exit
Choose an option: 2
Enter Reservation Number: AR71

Available Flights:
Flight Number: MEA101, Departure: BEIRUT, Destination: KUWAIT CITY, Available Seats: 5
Flight Number: TK102, Departure: ISTANBUL, Destination: BANGKOK, Available Seats: 3
Choose a flight (1 or 2): 1
Enter Customer ID: 1234
Enter Customer Name: Demo
Enter Customer Email: Demo@gmail.com

Reservation made successfully!
Reservation Number: AR71
Customer ID: 1234, Name: Demo, Email: Demo@gmail.com
Flight Number: MEA101, Departure: BEIRUT, Destination: KUWAIT CITY, Available Seats: 4
```

This part the user chooses option 2 to make a reservation. As soon as the user clicks on number 2, it asks the user to enter their reservation number and then it displays the available flights for the user to choose their ticket. After the user chooses, it asks them to enter its customer Id, name, and email. Finally, it displays their successful reservation. Note: it also decreases the available seats, in this case it was originally 5 and then it became 4.

```
=== Flight Reservation System ===
1. Display Flights
2. Make a Reservation
3. Search Reservation
4. Cancel Reservation
5. Display All Reservations
6. Display Canceled Reservations
7. Exit
Choose an option: 3
Enter Reservation Number to search: AR71
Reservation found!
```

This part the user enters 3 which demonstrates that they want to search their flight reservation. It asks the user to enter their reservation number and then displays whether it is found or not. The below case is if it is not found.

```
=== Flight Reservation System ===
1. Display Flights
2. Make a Reservation
3. Search Reservation
4. Cancel Reservation
5. Display All Reservations
6. Display Canceled Reservations
7. Exit
Choose an option: 3
Enter Reservation Number to search: TK98
Reservation not found.
```

```
=== Flight Reservation System ===
1. Display Flights
2. Make a Reservation
3. Search Reservation
4. Cancel Reservation
5. Display All Reservations
6. Display Canceled Reservations
7. Exit
Choose an option: 4
Enter Reservation Number to cancel: AR71
Reservation cancelled successfully!
```

This part the user enters option 4 which is to cancel their reservation.
It asks the user to enter its reservation number and then cancels it.

```
=== Flight Reservation System ===
1. Display Flights
2. Make a Reservation
3. Search Reservation
4. Cancel Reservation
5. Display All Reservations
6. Display Canceled Reservations
7. Exit
Choose an option: 5

All Reservations:
No reservations found.
```

Here the user enters option 5 which is to display all reservation but since the user has previously cancelled their reservation it displays not found. The below case is if there was a reservation.

```
=== Flight Reservation System ===
1. Display Flights
2. Make a Reservation
3. Search Reservation
4. Cancel Reservation
5. Display All Reservations
6. Display Canceled Reservations
7. Exit
Choose an option: 5

All Reservations:
Reservation Number: AR71
Customer ID: 1234, Name: Demo, Email: Demo@gmail.com
Flight Number: MEA101, Departure: BEIRUT, Destination: KUWAIT CITY, Available Seats: 4
```

```
=== Flight Reservation System ===
1. Display Flights
2. Make a Reservation
3. Search Reservation
4. Cancel Reservation
5. Display All Reservations
6. Display Canceled Reservations
7. Exit
Choose an option: 6

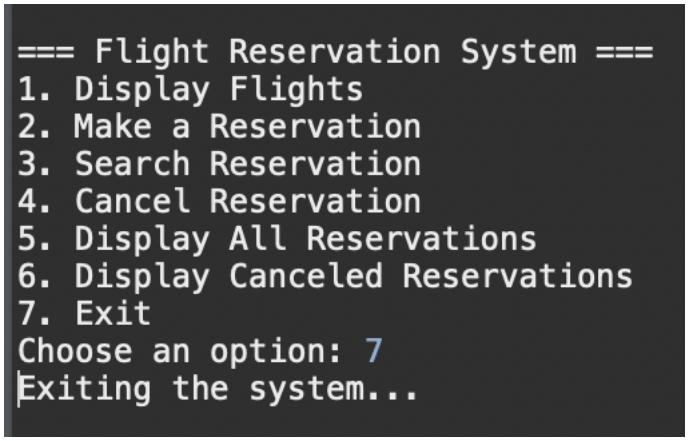
Canceled Reservations:
Reservation Number: AR71
Customer ID: 1234, Name: Demo, Email: Demo@gmail.com
Flight Number: MEA101, Departure: BEIRUT, Destination: KUWAIT CITY, Available Seats: 5
```

Here the user chooses option 6 which is to display all canceled reservation.
The below case is if there is no canceled reservation.

```
=== Flight Reservation System ===
1. Display Flights
2. Make a Reservation
3. Search Reservation
4. Cancel Reservation
5. Display All Reservations
6. Display Canceled Reservations
7. Exit
Choose an option: 6

Canceled Reservations:
No canceled reservations.
```

Finally, the below case is when the user wants to exit the system.



If the user enters a number that is not from 1-7 it shows:

